B E G I N N E R C A S E S T U D Y

BookStore

Spring Cloud Microservices — Full CRUD Edition

Book Service + Order Service | Eureka | API Gateway | Feign Client

application.properties only | No YAML | No Security | Explanation + Configuration Steps

2 Hours	Java 17+	Spring Boot 3	Spring Cloud	Full CRUD	No Security

Two services, 5 CRUD endpoints each, communicating through Feign Client

---

Section 1 — The Story

1.1 What We Are Building

BookStore is a small online bookshop. You are the developer tasked with building its backend from scratch using microservices. The entire system has two main services:

Service	Port	Responsibility	CRUD Operations	Talks To
Book Service	8081	Manages the book catalog — every book in the store	Create, Read (all + by id), Update, Delete a book	Nobody — it just stores books
Order Service	8082	Lets customers place and manage orders for books	Create, Read (all + by id), Update status, Delete an order	Book Service — via Feign Client

You also set up two infrastructure services to connect everything:

- Eureka Server — a phone book where all services register their address. Others look up addresses here instead of hard-coding them.
- API Gateway — the single door for all HTTP requests. Clients send all requests to port 8080, and the Gateway routes them to the correct service.

1.2 Strict Rules for This Case Study

IMPORTANT *These rules apply to every step in this document. Do not break them.*

Rule	What It Means
NO YAML anywhere	Every service must use application.properties only. No application.yml files — not even for the API Gateway.
NO Spring Security	There is no authentication or authorisation. All endpoints are open. Focus is on architecture and configuration.
NO code shown	This document explains what to create and configure. No Java code snippets. You write the Java yourself based on the descriptions.
application.properties first	Write all properties files before writing Java code. Correct configuration is more important than code.

Section 2 — UML Diagrams

> **NOTE** *Study all three diagrams before writing a single line of code. Understanding the architecture visually is the most important step for a beginner.*

2.1 System Architecture Diagram

This diagram shows all four services, their ports, the CRUD endpoints each service exposes, and how the Feign Client connects Order Service to Book Service.

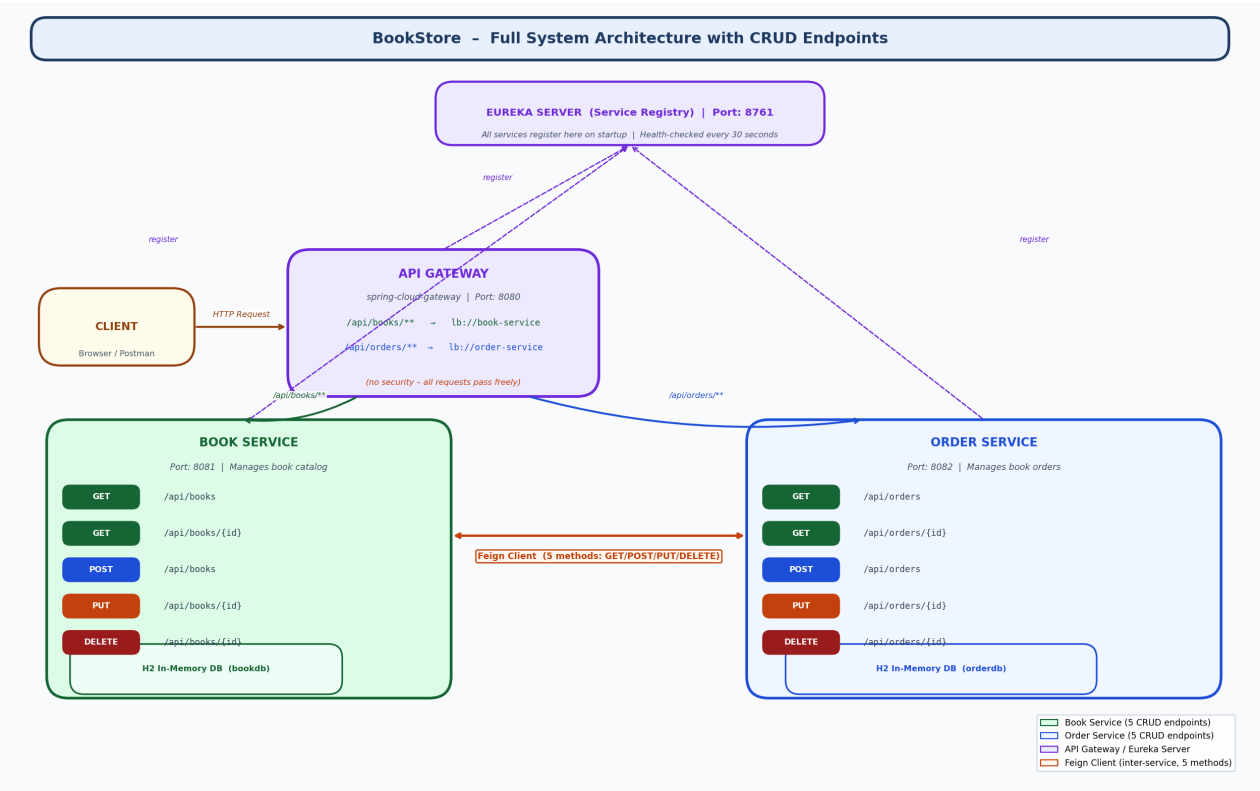

Figure 1 — Full System Architecture with All CRUD Endpoints

How to Read This Diagram

- **Green service (Book Service):** shows all 5 CRUD endpoints it exposes on port 8081
- **Blue service (Order Service):** shows all 5 CRUD endpoints it exposes on port 8082
- **Orange double arrow (Feign Client):** Order Service calls Book Service directly using 5 Feign methods. The client never sees this call.
- **Purple boxes (Gateway + Eureka):** Gateway routes all /api/books/** to Book Service and /api/orders/** to Order Service using lb:// URIs resolved by Eureka
- **Dashed arrows:** services register themselves with Eureka on startup. No manual registration needed.

2.2 Entity / Class Diagram

This diagram shows the exact fields inside each entity and the relationship between Book, Order, and BookDTO.

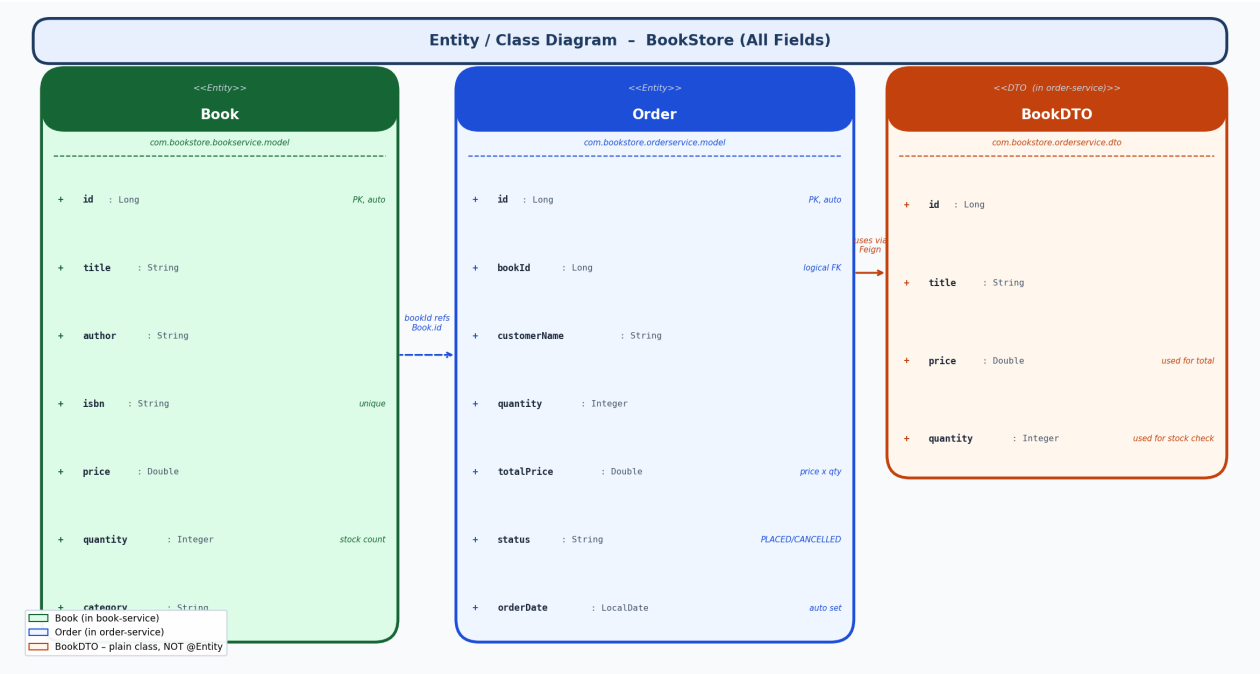

Figure 2 — Entity Diagram with All Fields (Book, Order, BookDTO)

Key Points From This Diagram

- **Book entity** lives only in Book Service. It has 7 fields. The id field is the primary key generated automatically by JPA.
- **Order entity** lives only in Order Service. It stores bookId as a plain Long — not a Book object. This is how microservices cross service boundaries.
- **BookDTO** is a plain Java class (NOT @Entity) inside Order Service. It mirrors only the 4 fields from Book that Order Service needs. Feign fills this automatically when calling GET /api/books/{id}.
- **The dashed arrow** between Order.bookId and Book.id shows a logical reference — there is no database foreign key because each service has its own separate database.

2.3 Sequence Diagram — Place a Book Order

This diagram shows the exact 9-step sequence that happens when a customer places an order. Every step that Feign and Eureka handle is shown.

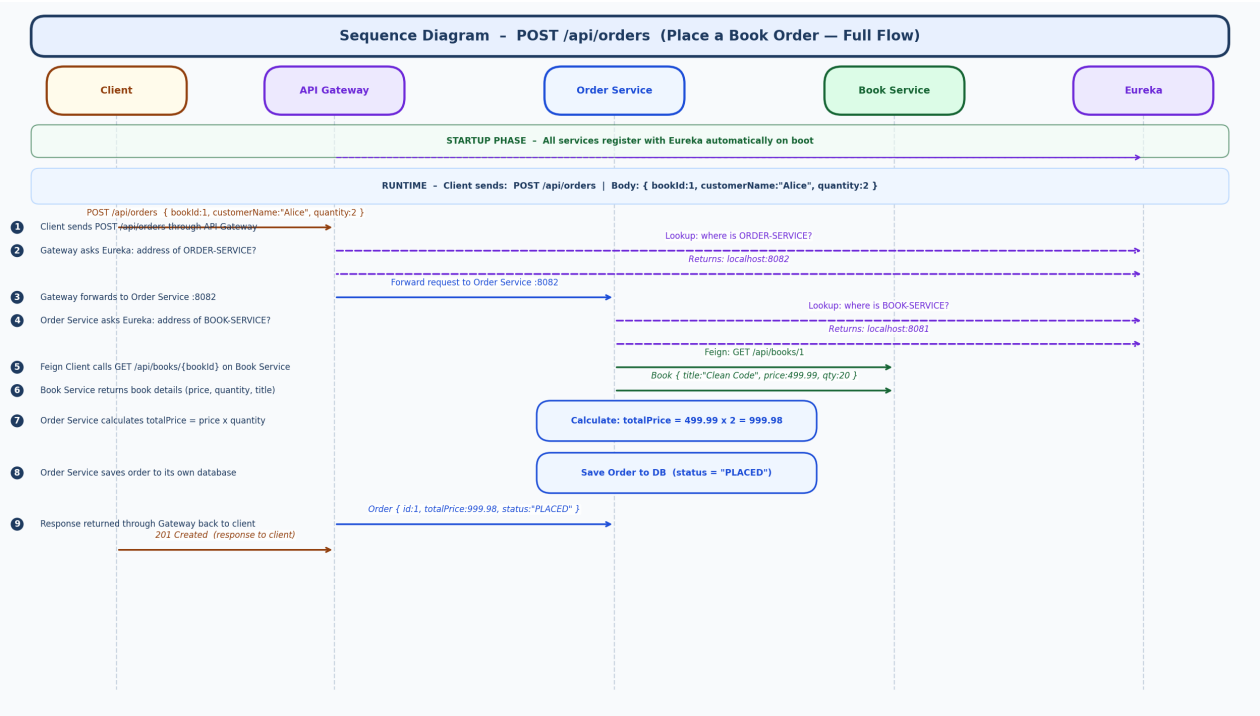

Figure 3 — Sequence Diagram: POST /api/orders (9 Steps)

Step	Who Acts	What Happens
1	Client → API Gateway	Client sends POST /api/orders with bookId, customerName, quantity
2	Gateway → Eureka	Gateway asks Eureka: what is the address of ORDER-SERVICE?
3	Eureka → Gateway	Eureka replies: ORDER-SERVICE is at localhost:8082
4	Gateway → Order Service	Gateway forwards the request to localhost:8082/api/orders
5	Order Service → Eureka	Order Service asks Eureka: what is the address of BOOK-SERVICE? (Feign does this automatically)
6	Eureka → Order Service	Eureka replies: BOOK-SERVICE is at localhost:8081
7	Order Service → Book Service	Feign calls GET /api/books/{bookId} on localhost:8081 to get book price
8	Book Service → Order Service	Book Service returns Book details: title, price, quantity
9	Order Service → Client (via Gateway)	Order Service calculates totalPrice, saves order, returns 201 Created

Section 3 — Key Concepts Explained

3.1 What is a Microservice?

A microservice is a small, focused application that:

- Does exactly ONE business job (book catalog OR order management — never both)
- Runs as a completely separate process on its own port
- Has its own isolated database — it never shares tables with another service
- Communicates with other services only through HTTP REST APIs
- Can be started, stopped, updated, and scaled completely independently

3.2 Service Registry and Discovery (Eureka)

> **CONCEPT** *Problem: If Order Service wants to call Book Service, it needs an address. If you hard-code "http://localhost:8081" in the code, it breaks the moment Book Service moves to a different machine or port. Service Discovery solves this permanently.*

What Happens	Who Does It	When
Book Service announces: "I am BOOK-SERVICE running at localhost:8081"	Book Service (automatic via @EnableDiscoveryClient)	At startup — every time Book Service boots
Order Service announces: "I am ORDER-SERVICE running at localhost:8082"	Order Service (automatic via @EnableDiscoveryClient)	At startup
API Gateway announces: "I am API-GATEWAY running at localhost:8080"	API Gateway (automatic via @EnableDiscoveryClient)	At startup
Eureka keeps a live registry of all registered services	Eureka Server	Constantly — sends a heartbeat check every 30 seconds
Feign asks: "Give me the address of BOOK-SERVICE right now"	Feign Client (inside Order Service)	Every time Order Service needs to call Book Service
Eureka replies: "BOOK-SERVICE is at localhost:8081"	Eureka Server	At the moment of the Feign call

3.3 API Gateway

The API Gateway is the single public entry point. No client ever calls Book Service or Order Service directly. They only call the Gateway on port 8080. The Gateway then:

- Matches the URL against route rules you configure in application.properties
- Looks up the real address from Eureka using the service name
- Forwards the request transparently — the client never knows the internal address

URL Client Sends to Gateway	Route Rule Matches	Forwarded To (via Eureka)
GET /api/books	Path=/api/books/**	Book Service at localhost:8081
GET /api/books/3	Path=/api/books/**	Book Service at localhost:8081/api/books/3
POST /api/books	Path=/api/books/**	Book Service at localhost:8081/api/books
PUT /api/books/2	Path=/api/books/**	Book Service at localhost:8081/api/books/2
DELETE /api/books/2	Path=/api/books/**	Book Service at localhost:8081/api/books/2
POST /api/orders	Path=/api/orders/**	Order Service at localhost:8082/api/orders
GET /api/orders	Path=/api/orders/**	Order Service at localhost:8082/api/orders
PUT /api/orders/1	Path=/api/orders/**	Order Service at localhost:8082/api/orders/1
DELETE /api/orders/1	Path=/api/orders/**	Order Service at localhost:8082/api/orders/1

NOTE *lb://book-service in the Gateway configuration means "load-balanced lookup". The lb:// prefix tells Spring Cloud Gateway to ask Eureka for the address of book-service at request time instead of using a hard-coded URL.*

3.4 Feign Client (Inter-Service Communication)

CONCEPT *Feign Client is the Spring Cloud way for one microservice to call another microservice using a clean, declarative Java interface. Instead of writing manual HTTP code, you define what you want to call, and Spring generates the actual HTTP request automatically.*

Feign Client in this project:

- Lives inside Order Service only. Book Service does not use Feign.
- The interface (BookClient) declares 5 methods — one for each CRUD endpoint in Book Service.
- When Order Service calls bookClient.getBookById(1L), Feign automatically builds a GET request to http://{book-service-address}/api/books/1.
- The address is resolved from Eureka using the name "book-service" — no hard-coded URL.
- The response JSON is automatically mapped into a BookDTO object.

Feign Method in BookClient	HTTP Call Made to Book Service	Return Type	When Order Service Uses It
getAllBooks()	GET /api/books	List<BookDTO>	When order wants to check all available books

Feign Method in BookClient	HTTP Call Made to Book Service	Return Type	When Order Service Uses It
getBookById(Long id)	GET /api/books/{id}	BookDTO	Before placing an order — to get book price
createBook(BookDTO book)	POST /api/books	BookDTO	If order service needs to add a book entry
updateBook(Long id, BookDTO book)	PUT /api/books/{id}	BookDTO	To update stock or price from order context
deleteBook(Long id)	DELETE /api/books/{id}	void	To remove a book via order service flow

IMPORTANT *@EnableFeignClients annotation MUST be on the OrderServiceApplication main class. If it is missing, Spring will not create a bean for the BookClient interface and you will get a NoSuchBeanDefinitionException error when Order Service starts.*

Section 4 — Creating the Projects

NOTE *Go to https://start.spring.io for each project. Use these settings for all: Project = Maven, Language = Java, Spring Boot = 3.2.x, Java = 17, Packaging = Jar.*

Step 1 — Eureka Server

Field	Value
Artifact	eureka-server
Description	Service Registry for BookStore
Dependencies to add	Eureka Server

Step 2 — Book Service

Field	Value
Artifact	book-service
Description	Manages the BookStore catalog
Dependency 1	Spring Web
Dependency 2	Spring Data JPA
Dependency 3	H2 Database
Dependency 4	Eureka Discovery Client
Dependency 5	Spring Boot Actuator

Step 3 — Order Service

Field	Value
Artifact	order-service
Description	Manages BookStore orders
Dependency 1	Spring Web
Dependency 2	Spring Data JPA
Dependency 3	H2 Database
Dependency 4	Eureka Discovery Client
Dependency 5	OpenFeign
Dependency 6	Spring Boot Actuator

IMPORTANT *Order Service needs BOTH Eureka Discovery Client AND OpenFeign. They are different dependencies. OpenFeign enables @FeignClient. Eureka Discovery Client enables registration and service lookup. Missing OpenFeign means @FeignClient interfaces will not work.*

Step 4 — API Gateway

Field	Value
Artifact	api-gateway
Description	Entry point for all BookStore requests
Dependency 1	Gateway
Dependency 2	Eureka Discovery Client

IMPORTANT *Do NOT add Spring Web to the api-gateway project. Spring Cloud Gateway uses a reactive engine (WebFlux). Adding Spring Web alongside Gateway causes a startup conflict. Your Gateway will fail to start if both are present in pom.xml.*

Section 5 — Entity Fields

NOTE *No code is shown. Create each class yourself using these field descriptions as the exact specification. Every field listed here must exist in the class.*

5.1 Book Entity — in Book Service

Package: com.bookstore.bookservice.model | Class name: Book | Annotate with: @Entity, @Table(name="books")

Field Name	Java Type	JPA Annotation / Note	Example Value
id	Long	@Id @GeneratedValue(strategy=IDENTITY) — primary key, auto-incremented	1, 2, 3 ...
title	String	@Column(nullable=false) — cannot be null	Clean Code
author	String	@Column(nullable=false)	Robert C. Martin
isbn	String	@Column(unique=true) — no two books share the same ISBN	978-0-13-235088-4
price	Double	@Column(nullable=false) — stored as decimal	499.99
quantity	Integer	@Column — number of copies in stock	25
category	String	@Column — genre or category label	Programming

TIP *Also generate: a no-argument constructor, an all-argument constructor, and getter/setter methods for every field. In Spring Boot 3.x with Lombok, use @Data @NoArgsConstructor @AllArgsConstructor on the class to generate all of these automatically.*

5.2 Order Entity — in Order Service

Package: com.bookstore.orderservice.model | Class name: Order | Annotate with: @Entity, @Table(name="orders")

Field Name	Java Type	JPA Annotation / Note	Example Value
id	Long	@Id @GeneratedValue(strategy=IDENTITY) — primary key, auto-incremented	1, 2, 3 ...
bookId	Long	@Column(nullable=false) — logical reference to Book.id. NOT a @ManyToOne — this is intentional in microservices.	1
customerName	String	@Column(nullable=false) — name of the person ordering	Alice
quantity	Integer	@Column(nullable=false) — how many copies ordered	2
totalPrice	Double	@Column — calculated by OrderService: book.price x quantity. Set programmatically, not by the client.	999.98
status	String	@Column — order lifecycle state. Set to "PLACED" on creation. Updated to "CANCELLED" if deleted.	PLACED
orderDate	LocalDate	@Column — set automatically to LocalDate.now() when order is saved. Client does not send this.	2025-03-10

> **NOTE** *totalPrice and orderDate are NOT sent by the client in the request body. They are calculated and set inside the OrderService class before saving. The client only sends: bookId, customerName, and quantity.*

5.3 BookDTO — in Order Service

Package: com.bookstore.orderservice.dto | Class name: BookDTO

This is a plain Java class. DO NOT add @Entity, @Table, or any JPA annotation. It is only used to hold the JSON response that Feign receives from Book Service.

Field Name	Java Type	Why Needed
id	Long	To know which book was returned
title	String	Can be stored in the order response for display purposes
price	Double	The only field used in the business logic: totalPrice = price x quantity
quantity	Integer	Used to check whether enough stock exists before placing the order

Section 6 — CRUD Operations (All Endpoints)

> **NOTE** *No code is shown. This section tells you the exact HTTP method, URL, request body, and expected response for every endpoint you must implement.*

6.1 Book Service — 5 CRUD Endpoints

All endpoints in Book Service follow the base path /api/books. Create a @RestController class in Book Service that exposes all 5 operations.

Method	URL	What It Does	Request Body	Response
GET	`/api/books`	Returns a list of all books in the database	None (no body needed)	200 OK + JSON array of all Book objects
GET	`/api/books/{id}`	Finds and returns one book by its ID	None	200 OK + single Book JSON or 404 Not Found if ID does not exist
POST	`/api/books`	Saves a new book into the database	JSON with title, author, isbn, price, quantity, category	201 Created + the saved Book JSON with the generated id
PUT	`/api/books/{id}`	Updates all fields of an existing book by ID	JSON with updated fields (same structure as POST body)	200 OK + the updated Book JSON

Method	URL	What It Does	Request Body	Response
DELETE	/api/books/{id}	Removes a book from the database by its ID	None	204 No Content (empty body — confirms deletion)

NOTE *For the GET by ID endpoint, use Optional<Book> from the repository and return 404 Not Found if the book does not exist. Spring provides ResponseEntity to return different HTTP status codes from the same method.*

6.2 Order Service — 5 CRUD Endpoints

All endpoints in Order Service follow the base path /api/orders. Create a @RestController class that exposes all 5 operations. The POST (create) operation MUST call Feign to fetch book details before saving.

Method	URL	What It Does	Request Body	Response
GET	/api/orders	Returns all orders in the database	None	200 OK + JSON array of all Order objects
GET	/api/orders/{id}	Finds and returns one order by its ID	None	200 OK + single Order JSON or 404 if not found
POST	/api/orders	Places a new order. Calls Feign to get book price. Calculates total. Saves order.	JSON: { bookId, customerName, quantity }	201 Created + saved Order JSON with totalPrice and status=PLACED
PUT	/api/orders/{id}	Updates the status of an existing order by ID (e.g., from PLACED to SHIPPED)	JSON: { status: "SHIPPED" }	200 OK + updated Order JSON
DELETE	/api/orders/{id}	Deletes/cancels an order by ID	None	204 No Content

CONCEPT *The POST /api/orders operation is the most important. The sequence is: (1) Receive request body. (2) Call bookClient.getBookById(request.getBookId()) via Feign. (3) Set order.setTotalPrice(book.getPrice() x request.getQuantity()). (4) Set status="PLACED" and orderDate=LocalDate.now(). (5) Save via repository and return 201.*

6.3 BookClient Interface — All 5 Feign Methods

Create an interface named BookClient in Order Service at package com.bookstore.orderservice.client. This interface mirrors every endpoint in Book Service. Do not implement any method — Spring generates the HTTP code automatically.

Annotation on Method	Method Signature	Maps to Book Service Endpoint	Return Type
@GetMapping("/api/books")	getAllBooks()	GET /api/books in Book Service	List<BookDTO>
@GetMapping("/api/books/{id}")	getBookById(@PathVariable Long id)	GET /api/books/{id} in Book Service	BookDTO
@PostMapping("/api/books")	createBook(@RequestBody BookDTO book)	POST /api/books in Book Service	BookDTO
@PutMapping("/api/books/{id}")	updateBook(@PathVariable Long id, @RequestBody BookDTO book)	PUT /api/books/{id} in Book Service	BookDTO
@DeleteMapping("/api/books/{id}")	deleteBook(@PathVariable Long id)	DELETE /api/books/{id} in Book Service	void

The interface also requires one class-level annotation above it:

Annotation	Value	Explanation
@FeignClient	name = "book-service"	"book-service" must exactly match the spring.application.name property in Book Service application.properties. Feign uses this name to look up the real address from Eureka.

IMPORTANT *The @FeignClient name is case-sensitive at the application level. If Book Service has spring.application.name=book-service, then @FeignClient must be name="book-service". Any mismatch causes Feign to fail with a 404 or connection error at runtime.*

Section 7 — Annotations on Main Application Classes

Each service needs specific annotations on its main class. These annotations configure the Spring Cloud role of each service.

Service	Main Class Name	Required Annotations	What Each Annotation Does
Eureka Server	EurekaServerApplication	@SpringBootApplication @EnableEurekaServer	@EnableEurekaServer — turns this into a service registry. All other services will register

Service	Main Class Name	Required Annotations	What Each Annotation Does
			themselves here on startup.
Book Service	BookServiceApplication	@SpringBootApplication @EnableDiscoveryClient	@EnableDiscoveryClient — makes this service auto-register with Eureka on startup. Spring reads spring.application.name to determine the registration key.
Order Service	OrderServiceApplication	@SpringBootApplication @EnableDiscoveryClient @EnableFeignClients	@EnableFeignClients — REQUIRED. Tells Spring to scan for all @FeignClient interfaces in the project and create working proxy beans for them.
API Gateway	ApiGatewayApplication	@SpringBootApplication @EnableDiscoveryClient	@EnableDiscoveryClient — Gateway must register with Eureka to use lb:// URIs for service lookup in its route configuration.

Section 8 — application.properties Configuration

IMPORTANT *This is the most critical section. Every property must be written exactly as shown. YAML is NOT used anywhere in this project — all services use application.properties files only.*

8.1 Eureka Server — application.properties

File location: eureka-server/src/main/resources/application.properties

```
# eureka-server/src/main/resources/application.properties
# Port configuration
server.port=8761

# Application name
spring.application.name=eureka-server

# CRITICAL: Prevent Eureka from registering with itself
# A standalone Eureka server must NOT act as a client to itself
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false

# The URL of this Eureka instance (used by all client services)
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

```
# Suppress startup warning logs
eureka.server.wait-time-in-ms-when-sync-empty=0
```

Property	Value	Why It Is Needed
server.port	8761	Eureka runs on 8761 by universal Spring Cloud convention. All client services expect this port by default.
spring.application.name	eureka-server	The name shown in the Eureka dashboard for this server entry.
register-with-eureka=false	false	Without this, Eureka Server tries to register itself as a client. It fails, prints repeated error warnings, and wastes startup time.
fetch-registry=false	false	Without this, Eureka tries to fetch a registry of services from itself. This creates a circular dependency error on startup.
defaultZone	http://localhost:8761/eureka/	The complete URL that client services must use to reach this Eureka server. The /eureka/ path suffix is mandatory.

8.2 Book Service — application.properties

File location: book-service/src/main/resources/application.properties

```
# book-service/src/main/resources/application.properties
# Server port
server.port=8081

# Application name — this MUST match @FeignClient(name="book-service") in Order
Service
spring.application.name=book-service

# H2 in-memory database
spring.datasource.url=jdbc:h2:mem:bookdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

# JPA — create table on startup, drop on shutdown
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true

# H2 web console — access at http://localhost:8081/h2-console
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# Eureka registration
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
eureka.client.register-with-eureka=true
```

```
eureka.client.fetch-registry=true

# Use IP address when registering with Eureka
eureka.instance.prefer-ip-address=true
```

Property	Value	Why It Is Needed
server.port	8081	Book Service must run on a unique port. Every service in the system needs a different port.
spring.application.name	book-service	This is the registration key in Eureka. The value must be exactly "book-service" — it must match the name attribute in @FeignClient in Order Service.
spring.datasource.url	jdbc:h2:mem:bookdb	Creates an H2 in-memory database named bookdb. The data exists only while the service is running. On restart, all data is cleared — this is fine for learning.
spring.jpa.hibernate.ddl-auto	create-drop	JPA reads your @Entity classes and automatically creates the books table at startup. The table is dropped when the service shuts down.
spring.h2.console.enabled	true	Enables a browser-based SQL console at http://localhost:8081/h2-console where you can query the bookdb database directly during development.
eureka.client.register-with-eureka	true	Book Service must register itself with Eureka so that Order Service and the Gateway can discover its address.
eureka.instance.prefer-ip-address	true	Registers using the numeric IP address (e.g. 127.0.0.1) instead of hostname. This prevents DNS resolution failures in some environments.

8.3 Order Service — application.properties

File location: order-service/src/main/resources/application.properties

```
# order-service/src/main/resources/application.properties
# Server port
server.port=8082

# Application name
spring.application.name=order-service

# H2 in-memory database — SEPARATE database from book-service
spring.datasource.url=jdbc:h2:mem:orderdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
```

```
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

# JPA
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true

# H2 web console — access at http://localhost:8082/h2-console
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# Eureka registration
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
eureka.client.register-with-eureka=true
eureka.client.fetch-registry=true
eureka.instance.prefer-ip-address=true
```

NOTE *Order Service uses its own separate database named orderdb. This is a fundamental microservices rule. The bookdb (in Book Service) and orderdb (in Order Service) cannot see each other's tables at all. There is no shared database.*

8.4 API Gateway — application.properties

File location: api-gateway/src/main/resources/application.properties

IMPORTANT: This file uses .properties format only — no YAML. The route configuration that was previously shown in YAML format is now written as flat key-value properties.

```
# api-gateway/src/main/resources/application.properties
# Server port
server.port=8080

# Application name
spring.application.name=api-gateway

# Enable Eureka-based automatic route discovery
spring.cloud.gateway.discovery.locator.enabled=true
spring.cloud.gateway.discovery.locator.lower-case-service-id=true

# ─── Route 1: Book Service ──────────────────────────────
# Route id — a unique name for this route (used in logs)
spring.cloud.gateway.routes[0].id=book-service-route
# lb:// means load-balanced — Eureka resolves "book-service" to the real address
spring.cloud.gateway.routes[0].uri=lb://book-service
# This route only applies to URLs starting with /api/books/
spring.cloud.gateway.routes[0].predicates[0]=Path=/api/books/**
# StripPrefix=0 means keep the full URL path when forwarding
spring.cloud.gateway.routes[0].filters[0]=StripPrefix=0

# ─── Route 2: Order Service ─────────────────────────────
spring.cloud.gateway.routes[1].id=order-service-route
spring.cloud.gateway.routes[1].uri=lb://order-service
spring.cloud.gateway.routes[1].predicates[0]=Path=/api/orders/**
spring.cloud.gateway.routes[1].filters[0]=StripPrefix=0

# Eureka connection
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
eureka.client.register-with-eureka=true
eureka.client.fetch-registry=true
```

```
eureka.instance.prefer-ip-address=true
```

Property Key	Value	Plain English Explanation
spring.cloud.gateway.routes[0].id	book-service-route	A unique identifier for this route rule. Used in debug logs when the Gateway processes a request. You can name it anything but it must be unique.
spring.cloud.gateway.routes[0].uri	lb://book-service	lb:// = load-balanced. "book-service" is looked up in Eureka to get the real address. This is how the Gateway avoids hard-coded URLs.
spring.cloud.gateway.routes[0].predicates[0]	Path=/api/books/**	This route rule only activates when the incoming URL starts with /api/books/. The ** wildcard matches any characters after it.
spring.cloud.gateway.routes[0].filters[0]	StripPrefix=0	Do not remove any path prefix before forwarding. With 0, the full path /api/books/3 is forwarded as-is to Book Service.
routes[1].id / uri / predicates / filters	order-service-route / lb://order-service / Path=/api/orders/**	Same pattern as the book route, but for Order Service. All /api/orders/** requests go to order-service.
discovery.locator.enabled	true	Allows the Gateway to automatically discover services registered in Eureka, even without explicit route configuration.
lower-case-service-id	true	Normalises service names to lowercase. Eureka registers as BOOK-SERVICE (uppercase), this setting allows the URL to use /book-service (lowercase).

NOTE *In properties format, array-style configuration uses [0], [1] indexes. routes[0] and routes[1] are two separate route configurations. This is exactly equivalent to writing - id: in YAML, but in flat key-value property format.*

Section 9 — Startup Order

IMPORTANT *This is the most common beginner mistake. Services MUST start in this exact order. Starting services in the wrong order causes registration failures and 503 errors.*

Order	Service	Port	Wait For This Before Starting Next	How to Confirm
1st	Eureka Server	8761	Eureka dashboard loads in browser	Open http://localhost:8761 — Eureka status page must appear before proceeding
2nd	Book Service	8081	BOOK-SERVICE appears in Eureka dashboard	Refresh http://localhost:8761 — look for BOOK-SERVICE under "Instances currently registered with Eureka"
3rd	Order Service	8082	ORDER-SERVICE appears in Eureka dashboard	Refresh http://localhost:8761 — ORDER-SERVICE must show status UP
4th	API Gateway	8080	All 3 services listed in Eureka as UP	Eureka must show: BOOK-SERVICE, ORDER-SERVICE, and API-GATEWAY all with status UP before testing

TIP *Run each service from a separate terminal window. Watch the console output as each starts. Look for the line "Registered instance BOOK-SERVICE with Eureka" — this confirms successful registration.*

Section 10 — Testing All CRUD Operations

NOTE *Always test through the API Gateway on port 8080. This proves the full flow — Gateway routing, Eureka discovery, and Feign communication — is working correctly.*

10.1 Book Service CRUD Tests

Test all 5 operations on Book Service through the Gateway. Start by creating a book, then use its returned id for the remaining tests.

#	Test Name	Method + URL (via Gateway)	Request Body	Expected Response
1	Create a book	POST http://localhost:8080/api/books	{ "title":"Clean Code", "author":"Robert C. Martin", "isbn":"978-0-13-235088-4", "price":499.99, "quantity":20, "category":"Programming" }	201 Created + Book JSON with "id":1
2	Get all books	GET http://localhost:8080/api/books	None	200 OK + JSON array containing

#	Test Name	Method + URL (via Gateway)	Request Body	Expected Response
				the book you just created
3	Get one book	GET http://localhost:8080/api/books/1	None	200 OK + single Book JSON for id=1
4	Update a book	PUT http://localhost:8080/api/books/1	{ "title":"Clean Code", "author":"Robert C. Martin", "isbn":"978-0-13-235088-4", "price":549.99, "quantity":18, "category":"Programming" }	200 OK + Book JSON with updated price=549.99
5	Delete a book	DELETE http://localhost:8080/api/books/1	None	204 No Content (empty body)

NOTE *After running test 5 (DELETE), re-create the book using test 1 before running Order Service tests. Order Service will call Feign to fetch book details — it needs the book to exist.*

10.2 Order Service CRUD Tests (with Feign)

Make sure at least one book exists before running these tests. The POST order test uses Feign to call Book Service internally.

#	Test Name	Method + URL (via Gateway)	Request Body	Expected Response
1	Place an order	POST http://localhost:8080/api/orders	{ "bookId":1, "customerName":"Alice", "quantity":2 }	201 Created + Order JSON with totalPrice (price x 2) and status="PLACED"
2	Get all orders	GET http://localhost:8080/api/orders	None	200 OK + JSON array of all orders
3	Get one order	GET http://localhost:8080/api/orders/1	None	200 OK + single Order JSON for id=1
4	Update order status	PUT http://localhost:8080/api/orders/1	{ "status":"SHIPPED" }	200 OK + Order JSON with status="SHIPPED"
5	Delete an order	DELETE http://localhost:8080/api/orders/1	None	204 No Content

CONCEPT *Test 1 (POST /api/orders) is the most important test. When it returns with a correct totalPrice (e.g. 999.98 for 2 x 499.99), it proves that: Feign Client connected to Book Service successfully, Eureka resolved the address correctly, and the business logic calculated the total correctly.*

Section 11 — Troubleshooting Guide

Problem	Root Cause	Solution
Service fails: "Connection refused localhost:8761"	Eureka Server is not running	Always start Eureka Server first. Wait for the dashboard at :8761 to appear before starting other services.
"NoSuchBeanDefinitionException: BookClient"	@EnableFeignClients is missing from OrderServiceApplication main class	Add @EnableFeignClients annotation directly above the class declaration in OrderServiceApplication.java
Gateway returns 503 for /api/books/**	Book Service not registered in Eureka, or not running	Check http://localhost:8761 — BOOK-SERVICE must show status UP. Restart Book Service if missing.
"Could not resolve host: book-service" from Feign	spring.application.name in Book Service does not match @FeignClient name	Check both: spring.application.name=book-service in Book Service properties AND @FeignClient(name="book-service") in BookClient interface. They must match exactly.
Gateway application.properties properties are ignored	File has wrong location or wrong format	Confirm the file is at api-gateway/src/main/resources/application.properties. Check that routes[0] indexing syntax is correct.
POST /api/orders returns 500 Internal Server Error	Feign call to Book Service failed — book may not exist	Create the book first using POST /api/books. The bookId in the order request must exist in Book Service database.
H2 console shows empty orders or books table	create-drop drops tables on restart	Re-add data after every restart. H2 in-memory data does not persist. This is expected behaviour.
Gateway fails to start with "Spring MVC found on classpath"	Spring Web dependency was added alongside Gateway	Remove the Spring Web dependency from api-gateway/pom.xml. Gateway uses WebFlux — it conflicts with Spring Web.

Quick Reference Card

Services at a Glance

Service	Port	Main Annotation(s)	spring.application.name	Registers in Eureka
Eureka Server	8761	@EnableEurekaServer	eureka-server	No — it IS Eureka
Book Service	8081	@EnableDiscoveryClient	book-service	Yes

Service	Port	Main Annotation(s)	spring.application.name	Registers in Eureka
Order Service	8082	@EnableDiscoveryClient + @EnableFeignClients	order-service	Yes
API Gateway	8080	@EnableDiscoveryClient	api-gateway	Yes

Feign Client — All 5 Methods at a Glance

Feign Method	Annotation	HTTP Call to Book Service	Used When
getAllBooks()	@GetMapping("/api/books")	GET /api/books	Browsing available books from Order Service
getBookById(Long id)	@GetMapping("/api/books/{id}")	GET /api/books/{id}	Fetching price before placing an order
createBook(BookDTO)	@PostMapping("/api/books")	POST /api/books	Adding a book via Order Service context
updateBook(Long, BookDTO)	@PutMapping("/api/books/{id}")	PUT /api/books/{id}	Updating book stock after an order
deleteBook(Long id)	@DeleteMapping("/api/books/{id}")	DELETE /api/books/{id}	Removing a book via Order Service context

Gateway Routes (properties format summary)

Property	Value
spring.cloud.gateway.routes[0].id	book-service-route
spring.cloud.gateway.routes[0].uri	lb://book-service
spring.cloud.gateway.routes[0].predicates[0]	Path=/api/books/**
spring.cloud.gateway.routes[0].filters[0]	StripPrefix=0
spring.cloud.gateway.routes[1].id	order-service-route
spring.cloud.gateway.routes[1].uri	lb://order-service
spring.cloud.gateway.routes[1].predicates[0]	Path=/api/orders/**
spring.cloud.gateway.routes[1].filters[0]	StripPrefix=0

Read the diagrams. Write the properties. Then write the code. *Configuration connects everything.*